A PRACTICAL COURSE

Async PostgreSQL in Rust

tokio-postgres from the first query to a production-shaped axum service — for developers who know Rust, a little Tokio and some SQL.

1 chapters · every runnable example was compiled and run against PostgreSQL as written

python3 tools/book.py book

Contents

0 Introduction and setup **1**
Setting up 1
A ten-line taste 2

CHAPTER 0

Introduction and setup

This book teaches `tokio-postgres`, the asynchronous PostgreSQL driver that most of Rust's database ecosystem stands on. It takes you from the first query to a production-shaped HTTP service built with axum, and at every step it shows what the driver sends over the wire, what the server does with it, and where the sharp edges are.

TODO: the rest of the introduction, from the outline in PLAN.md §7 (who the book is for, where tokio-postgres sits among Rust's Postgres libraries, how chapters are built, stockroom in one page).

Setting up

TODO: installing PostgreSQL and psql, `DATABASE_URL`, and creating the playground.

The playground is a scratch crate in which you run the book's examples. This is its `Cargo.toml`: the crates every runnable example in the book uses. The versions are minimums, so any newer compatible release works.

```toml
[package]
name = "playground"
version = "0.1.0"
edition = "2024"
publish = false

[dependencies]
anyhow = "1.0.104"
async-stream = "0.3.6"
axum = "0.8.9"
bytes = "1.12.1"
deadpool-postgres = "0.14.2"
futures-util = "0.3.34"
http-body-util = "0.1.5"
jiff = { version = "0.2.39", features = ["serde"] }
postgres-types = { version = "0.2.14", features = ["derive"] }
rust_decimal = { version = "1.43.0", features = ["db-tokio-postgres"] }
serde = { version = "1.0.229", features = ["derive"] }
serde_json = "1.0.154"
thiserror = "2.0.21"
tokio = { version = "1.54.1", features = ["full"] }
tokio-postgres = { version = "0.7.18", features = ["with-jiff-0_2", "with-serde_json-1",
"with-uuid-1", "array-impls"] }
tower = { version = "0.5.3", features = ["util"] }
uuid = { version = "1.29.0", features = ["v7", "serde"] }
```

A ten-line taste

Connect, send one query with a parameter, and read two typed values out of the row. Every piece of this is explained in Chapters 1 to 3. `connect` is the playground's helper, and Chapter 1 builds it line by line.

```rust
use playground::connect;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = connect().await?;

    // `$1` is a parameter: its value travels separately from the SQL text.
    let row = client
        .query_one(
            "SELECT $1::text || ', ' || current_user AS greeting, 6 * 7 AS answer",
            &[&"hello"],
        )
        .await?;

    let greeting: &str = row.get("greeting");
    let answer: i32 = row.get("answer");
    println!("{greeting}; the answer is {answer}");
    assert!(greeting.starts_with("hello, "));
    assert_eq!(answer, 42);
    Ok(())
}
```